CAS2105 Homework 6: Fake vs. Real News AI Pipeline

Seulbit Park (2022148099)

1 Introduction

Ever since the emergence of faster internet and personalized algorithms, the ability to differentiate fake, biased news has become exponentially more difficult. This project attempts to provide a solution through a simple text classification AI pipeline, trained by a Kaggle dataset consisting of fake and true news articles.

2 Task Definition

- **Task description:** Classify news article into "Fake" or "True".
- **Motivation:** Personal difficulty in differentiating fake and true news. Importance of absorbing unbiased, truthfully reported events.
- **Input / Output:** Model takes in 250 lines of "Fake.csv" and 250 lines of "True.csv" from Kaggle. Outputs the classification report that includes precision, recall, f1-score, support, and accuracy values. Also includes three examples of tested articles, the naive baseline's prediction, the AI model's prediction, and the actual classification.
- **Success criteria:** How high the precision and recall score of the model is.

3 Methods

This section includes both the naïve baseline and the improved AI pipeline.

3.1 Naïve Baseline

Fake vs. Real News Hueristic Approach

- **Method description:** After observing the csv files, articles in the "Fake.csv" had strong wording and were emphasized through capital letters. So, I counted the actual number of uppercase letters in one of the articles, and it came out to be around 5-10% in the article. Therefore, the naive baseline approach I took was to go through the entire article and count how many uppercase letters there are. If that is more than 5% of the full article length, that would be classified as fake news. If not, it would be considered true news.
- **Why naïve:** This is considered naive since it is merely following a keyword rule.
- **Likely failure modes:** If a true article happens to have more than five percent of uppercase letters (which often, they do!), then it would automatically be classified as fake news.

3.2 AI Pipeline

DistilBERT AI Pipeline for Text Classification

- **Models used:** distilbert-base-uncased
- **Pipeline stages:**

  1) Preprocessing: First, split the strings to remove "(Reuters)," which was a major indicator that an article was True. Converted strings into tokens using the tokenizer and modified them to all have length 128, so there is not too much data being inputted.

  2) Embedding/Representation: The model receives the tokenized inputs and uses the self-attention configuration to create embeddings.

  3) Decision/Ranking component: The transformer uses the linear layer to take the model's output and classify it into 0 (Fake) or 1 (True).

- **Design choices and justification:** I used the DistilBERT-uncased model as it is a common model used for text classification. It is related to the BERT model as it retains 97% of its performance despite being 40% smaller and 60% faster.

4 Experiments

4.1 Datasets

Dataset Description

- **Source:** Kaggle ISOT Fake News Dataset
- **Total examples:** 250 lines of Fake.csv and 250 lines of True.csv
- **Train/Test split:** I assigned 80% of the data to be used as training and 20% of data to be used for testing. I randomized the lines of Fake and True articles so there is no specific pattern of classification. I maintained the random state as 42, so the experiement is constant each run.
- **Preprocessing steps:**

  1) Modifying: Split string to get rid of "Reuters".

  2) Tokenization: Converted words into ID token numbers.

  3) Truncation: Made all article lengths to be 128.

  4) Normalization: Made all articles lowercase.

4.2 Metrics

Used "classification report" template from sklearn.

- **Classification:** precision, recall, F1, support

4.3 Results

Table 1: Naive Baseline Classification Results

Class	Precision	Recall	F1-Score	Support
Fake	0.67	0.19	0.29	43
True	0.60	0.93	0.73	57
Accuracy			**0.61**	100

Table 2: DistilBERT Pipeline Classification Results

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Fake	1.00	0.81	0.90	43
True	0.88	1.00	0.93	57
Accuracy			**0.92**	100

5 Reflection and Limitations

In the beginning, I had 500 lines of Fake.csv and 500 lines of True.csv, but that took an extremely long time to compute, so I changed the dataset to take in 250 lines of Fake.csv and True.csv each. Although the dataset decreased by half, the AI model still performed with 100% precision for Fake news classification and had a total accuracy of 92%.

Coming up with the naive baseline standard was the most difficult as I could not figure out a way to accurately identify fake articles. I definitely felt that only an AI model that can study the linguistic patterns of true articles would be able to identify the fake articles.

For metrics, the recall and precision scores convey the quality of the models most accurately. For example, the overall accuracy for the baseline model is 0.61 despite the fact that it only has a 0.19 recall for fake articles. I do not think the overall accuracy score (especially for the baseline) says much about the results.

Next time, I would like to train the model on a larger dataset and try to increase the recall score for Fake classification and precision for True classification.

References

Addoun, Mohammed. "Lab05_FakeNewsDetection." *Kaggle*, Kaggle, 26 Nov. 2025, www.kaggle.com/code/mohammedaddoun/lab05-fakenewsdetection.

"Bert & Distilbert: Efficient NLP Transformers." *Trending Papers*, www.emergentmind.com/topics/bert-distilbert.

"Classification Report ." *Classification Report - Yellowbrick v1.5 Documentation*, www.scikit-yb.org/en/latest/api/classifier/classification_report.html

"Distilbert." *DistilBERT*, huggingface.co/docs/transformers/en/model_doc/distilbert.

Pandey, Sandeep. "Text Classification Using Custom Data and Pytorch | by Sandeep Pandey | Medium." *Medium*, medium.com/@spandey8312/text-classification-using-custom-data-and-pytorch-d88ba1087045.